

Linux命令行与bash脚本

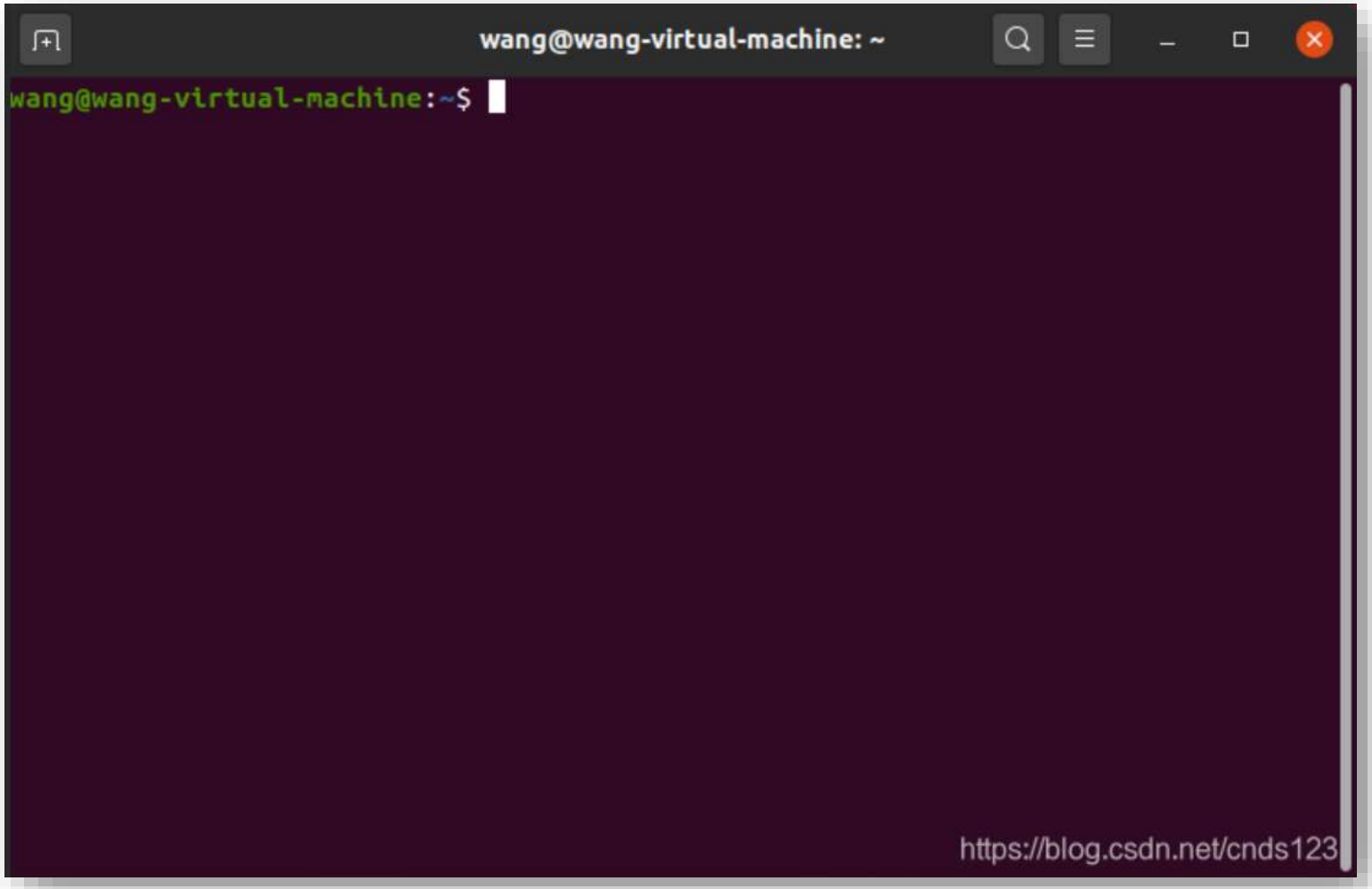


01
Linux命令行

包管理系统及版本控制系统

包管理器 和版本控制系统	作用
dpkg	安装本地deb格式软件
apt	从网上安装软件，同时管理下载的软件
git	帮助开发者高效、安全地管理代码（或其他文本文件）的历史变更
pip	用于安装和管理用 Python 编写的第三方库

shell



快捷键为

Ctrl+alt+t

Bash提示符

```
<username>@<hostname><current working directory>$
```

用户的主目录用~标记，是我们登录时的**默认文件夹**。

#代表以root登录，

$代表以用户登录。

Linux命令提示符在终端中的显示方式由**PS1变量来控制**，具体实现见

https://zhuanlan.zhihu.com/p/340003720

导航

pwd	显示当前所在位置
ls	显示当前所在位置有哪些文件和目录
cd	进入某个目录

使用文件和目录

touch	创建一个文件
mkdir	创建一个文件夹
tree	以树状（缩进+连线）的形式直观地显示目录和文件的层级结构
mv	移动文件或更改文件名
cp	复制文件或文件夹
rm	删除文件或文件夹

编辑查看文件以及更改权限

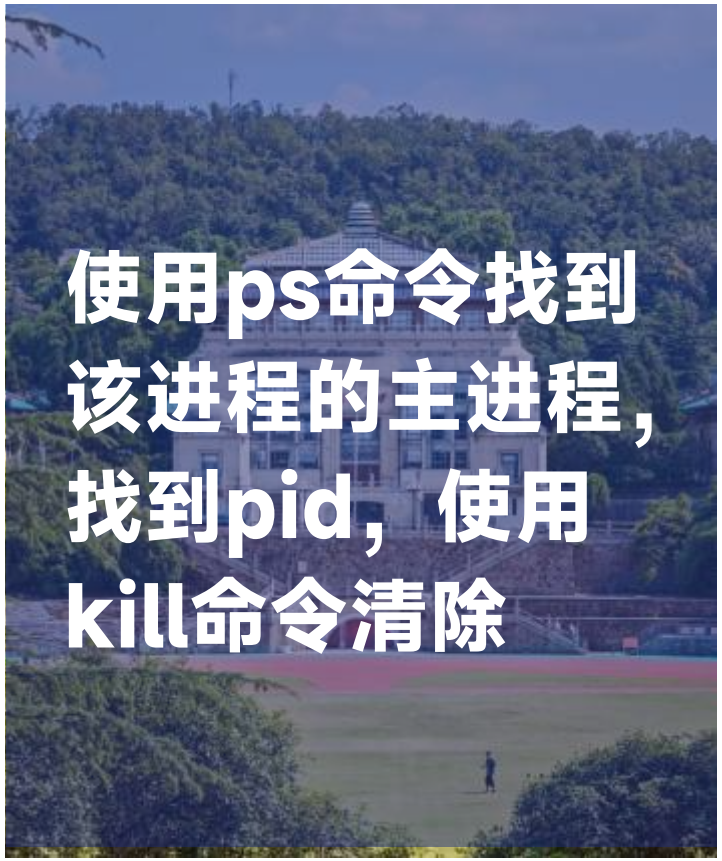
nano	**简单易用**的命令行文本编辑器（新手友好）
vim	**功能强大、高度可定制**的命令行文本编辑器（vimtutor教程）
cat	**查看**文件
chmod	**修改**文件或目录权限

处理陌生命令



- ✓ 首先百度；
- ✓ 深入研究具体用法使用man，-h，--help等

清除一个进程



- **a** 显示**所有终端上的进程**（不仅限于当前用户的）
- **u** 以**用户友好**的格式显示（包含 CPU、内存、TTY 等）
- **x** 显示**没有控制终端**的进程（如后台服务、守护进程）



02
Bash脚本

Bash脚本

```bash
#!/bin/bash

# Check for given arguments
if [ $# -eq 0 ]
then
    echo -e "You need to specify the target domain.\n"
    echo -e "Usage:"
    echo -e "\t$0 <domain>"
    exit 1
else
    domain=$1
fi
```

Bash脚本

```
#!/bin/bash

# Check for given arguments
if [ $# -eq 0 ]
then
    echo -e "You need to specify the target domain.\n"
    echo -e "Usage:"
    echo -e "\t$0 <domain>"
    exit 1
else
    domain=$1
fi
```

#! 是一个约定的标记，它告诉系统这个脚本需要什么解释器来执行，**即使用哪一种 Shell**，后面跟上shell的路径。

Bash脚本

```
#!/bin/bash

# Check for given arguments
if [ $# -eq 0 ]
then
    echo -e "You need to specify the target domain.\n"
    echo -e "Usage:"
    echo -e "\t$0 <domain>"
    exit 1
else
    domain=$1
fi
```

[是命令(test)，]为了对称美，注意空格

Bash脚本

```bash
#!/bin/bash

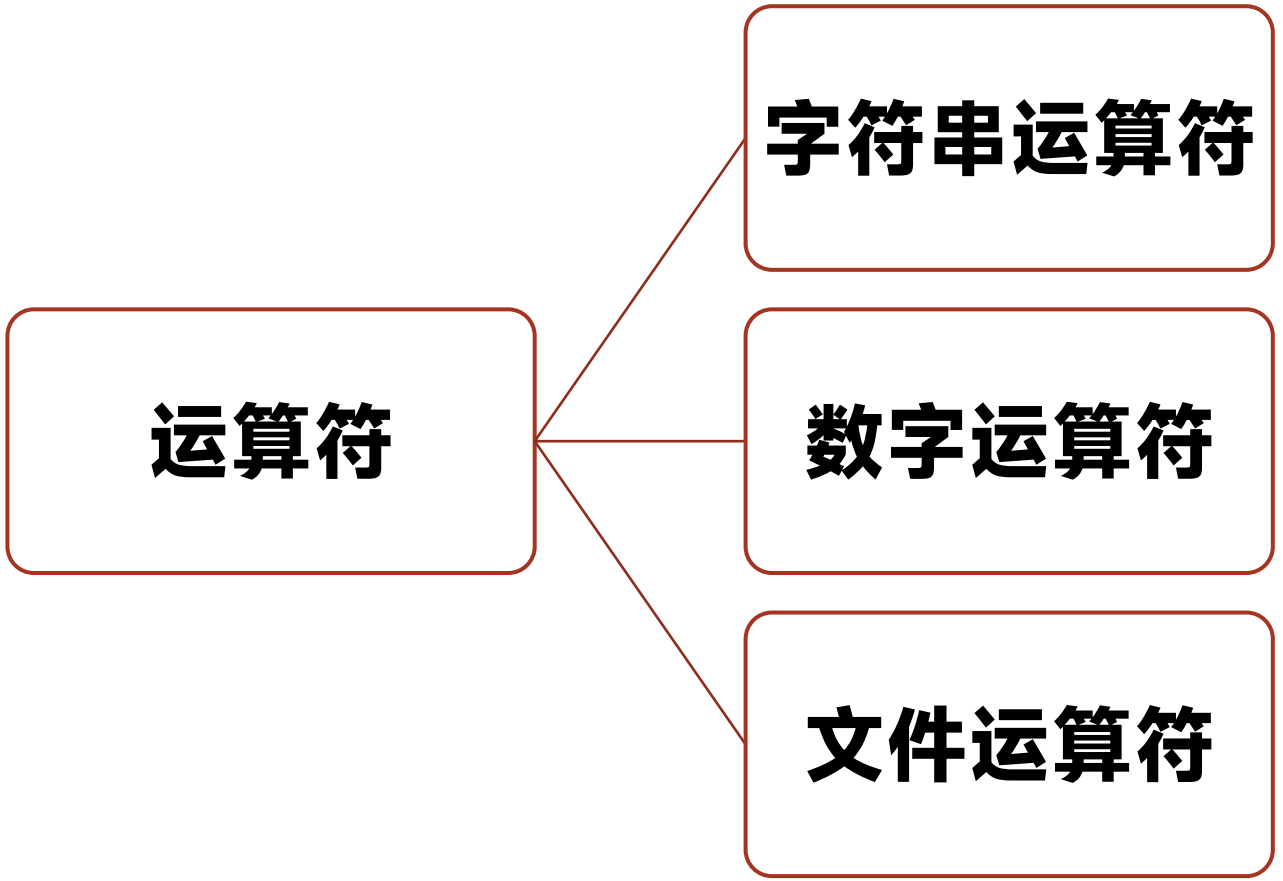
# Check for given arguments
if [ $# -eq 0 ]
then
    echo -e "You need to specify the target domain.\n"
    echo -e "Usage:"
    echo -e "\t$0 <domain>"
    exit 1
else
    domain=$1
fi
```

退出状态码，0为运行成功

参数可以有$0~9

Bash脚本

$#	This variable holds the number of arguments passed to the script.
$@	This variable can be used to retrieve the list of command-line arguments.
$n	Each command-line argument can be selectively retrieved using its position. For example, the first argument is found at $1.
$$	The process ID of the currently executing process.
$?	The exit status of the script. This variable is useful to determine a command's success. The value 0 represents successful execution, while 1 is a result of a failure.


运算符
字符串运算符
数字运算符
文件运算符

字符串运算符

Operator	Description
==	is equal to
!=	is not equal to
<	is less than in ASCII alphabetical order
>	is greater than in ASCII alphabetical order
-z	if the string is empty (null)
-n	if the string is not null

数字运算符

Operator	Description
-eq	is equal to
-ne	is not equal to
-lt	is less than
-le	is less than or equal to
-gt	is greater than
-ge	is greater than or equal to

文件运算符

Operator	Description
-e	if the file exist
-f	tests if it is a file
-d	tests if it is a directory
-L	tests if it is if a symbolic link
-N	checks if the file was modified after it was last read
-O	if the current user owns the file
-G	if the file's group id matches the current user's
-s	tests if the file has a size greater than 0
-r	tests if the file has read permission
-w	tests if the file has write permission
-x	tests if the file has execute permission

Bash脚本

$#	This variable holds the number of arguments passed to the script.
$@	This variable can be used to retrieve the list of command-line arguments.
$n	Each command-line argument can be selectively retrieved using its position. For example, the first argument is found at $1.
$$	The process ID of the currently executing process.
$?	The exit status of the script. This variable is useful to determine a command's success. The value 0 represents successful execution, while 1 is a result of a failure.

Bash脚本

```bash
# Identify IP address of the specified domain
hosts=$(host $domain | grep "has address" | cut -d" " -f4 | tee discovered_hosts.txt)

echo -e "Discovered IP address:\n$hosts\n"
ipaddr=$(host $domain | grep "has address" | cut -d" " -f4 | tr "\n" " ")

# Available options
echo -e "Additional options available:"
echo -e "\t1) Identify the corresponding network range of target domain."
echo -e "\t2) Ping discovered hosts."
echo -e "\t3) All checks."
echo -e "\t*) Exit.\n"

read -p "Select your option: " opt

case $opt in
    "1") network_range ;;
    "2") ping_host ;;
    "3") network_range && ping_host ;;
    "*") exit 0 ;;
esac
```

管道的作用

“|”是**Linux管道命令操作符**，简称**管道符**。使用此管道符“|”可以将两个命令分隔开，“|”左边命令的输出就会作为“|”右边命令的输入。

此命令可**连续使用**，第一个命令的输出会作为第二个命令的输入，第二个命令的输出又会作为第三个命令的输入，依此类推。

Bash脚本

```
# Identify IP address of the specified domain
hosts=$(host $domain | grep "has address" | cut -d" " -f4 | tee discovered_hosts.txt)

echo -e "Discovered IP address:\n$hosts\n"
ipaddr=$(host $domain | grep "has address" | cut -d" " -f4 | tr "\n" " ")

# Available options
echo -e "Additional options available:"
echo -e "\t1) Identify the corresponding network range of target domain."
echo -e "\t2) Ping discovered hosts."
echo -e "\t3) All checks."
echo -e "\t*) Exit.\n"

read -p "Select your option: " opt

case $opt in
    "1") network_range ;;
    "2") ping_host ;;
    "3") network_range && ping_host ;;
    "*") exit 0 ;;
esac
```

Case语法

case "字符串变量" in

值1) 指令1... ;;

值2) 指令2...;;

*) 指令3...esac

Bash脚本

```
# Identify Network range for the specified IP address(es)
function network_range {
    for ip in $ipaddr
    do
        netrange=$(whois $ip | grep -i "inetnum" | tee -a inetnum.txt)
        starts=$(whois $ip | grep -i "inetnum" | awk '{print $2}')
        ends=$(whois $ip | grep -i "inetnum" | awk '{print $4}')
        cidr_ips=$(prips "$starts" "$ends" | head -n 2)
        echo -e "\nNetRange for $ip:"
        echo -e "$netrange"
    done
}
```

函数语法

[function] funname

[()]

{

action;

[return int;]

}

Bash脚本

```bash
# Identify Network range for the specified IP address(es)
function network_range {
    for ip in $ipaddr
    do
        netrange=$(whois $ip | grep -i "inetnum" | tee -a inetnum.txt)
        starts=$(whois $ip | grep -i "inetnum" | awk '{print $2}')
        ends=$(whois $ip | grep -i "inetnum" | awk '{print $4}')
        cidr_ips=$(prips "$starts" "$ends" | head -n 2)
        echo -e "\nNetRange for $ip:"
        echo -e "$netrange"
    done
}
```

For语法

```
for var in item1 item2 ... itemN
do
    command1
    command2
    ...
    commandN
done
```

Bash脚本

```
# Ping discovered IP address(es)
function ping_host {
    hosts_up=0
    hosts_total=0

    echo -e "\nPinging host(s):"
    for host in $cidr_ips
    do
        stat=1
        while [ $stat -eq 1 ]
        do
            ping -c 2 $host > /dev/null 2>&1
            if [ $? -eq 0 ]
            then
                echo "$host is up."
                ((stat--))
                ((hosts_up++))
                ((hosts_total++))
            else
                echo "$host is down."
                ((stat--))
                ((hosts_total++))
            fi
        done
    done

    echo -e "\n$hosts_up out of $hosts_total hosts are up."
}
```

任务

编写一个bash脚本，实现多轮文本混淆，显示结果。

调用格式：./filename.sh <操作序列> <文件.txt>

支持的操作（大写）：

R：反转字符串

U：将所有字母转为大写

L：将所有字母转为小写

B：对文本进行base64编码

D：对文本进行base64解码

操作**从左到右**依次执行。

要求：

1、检验参数数量是否正确；

2、操作序列是否有非法字符（正则表达式）；

3、5个操作的功能均可实现。



Linux命令行与bash脚本